Supplementary Material-A Spectral Heterogeneous Diffusion Framework for Knowledge-aware Recommendation

Anonymous Author(s)

ACM Reference Format:
Anonymous Author(s). 2018. Supplementary Material-A Spectral Heterogeneous Diffusion Framework for Knowledge-aware Recommendation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 2 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Theoretical Analysis

1.1 Derivation of Equation (2)

To analytically solve the forward diffusion subprocess in our method, we start with the conventional heat equation. The heat equation is given by:

$$\frac{\partial \mathbf{x}}{\partial \tau} = \alpha \nabla^2 \mathbf{x} \tag{1}$$

where $\mathbf{x}(\tau)$ represents the state of the knowledge graph at time τ, α is a hyperparameter controlling the rate of diffusion, and ∇^2 is the Laplacian operator, representing the second-order derivative of $\mathbf{x}$. Next, we perform the spectral decomposition of the Laplacian operator. The Laplacian operator is expressed as:

$$\nabla^2 = \mathbf{A} - \mathbf{I} \tag{2}$$

where $\mathbf{A}$ is the item-item similarity matrix, and $\mathbf{I}$ is the identity matrix. We decompose the adjacency matrix $\mathbf{A}$ using its eigenvalue decomposition:

$$\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T \tag{3}$$

where $\mathbf{U} = [\mathbf{u}_1, \ldots, \mathbf{u}_l]$ represents the eigenvector matrix of $\mathbf{A}$, and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \ldots, \lambda_l)$ denotes the eigenvalue matrix of $\mathbf{A}$. Thus, the Laplacian operator $\nabla^2 = \mathbf{A} - \mathbf{I}$ can be written as:

$$\nabla^2 = \mathbf{U}(\mathbf{\Lambda} - \mathbf{I})\mathbf{U}^T \tag{4}$$

Substituting this spectral decomposition into the heat equation, we get:

$$\frac{\partial \mathbf{x}}{\partial \tau} = \alpha \mathbf{U}(\mathbf{\Lambda} - \mathbf{I})\mathbf{U}^T \mathbf{x} \tag{5}$$

To solve this differential equation, we separate variables and integrate with respect to time τ. The general solution to this type of equation is:

$$\mathbf{x}(\tau) = \exp\left(-\tau\alpha(\mathbf{I} - \mathbf{A})\right)\mathbf{x}(0) \tag{6}$$

Using the spectral decomposition $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$, this becomes:

$$\mathbf{x}(\tau) = \exp\left(-\tau\alpha(\mathbf{I} - \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T)\right)\mathbf{x}(0) \tag{7}$$

Simplifying further, we use the fact that $\mathbf{U}^T\mathbf{U} = \mathbf{I}$, leading to:

$$\mathbf{x}(\tau) = \mathbf{U}\left[\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{\Lambda})\right) \odot \left(\mathbf{U}^T\mathbf{x}(0)\right)\right] \tag{8}$$

Finally, we arrive at the following expression:

$$\mathbf{x}(\tau) = \mathbf{U}\left[\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{\Lambda})\right) \odot \left(\mathbf{U}^T\mathbf{x}(0)\right)\right] \tag{9}$$

where $\mathbf{U}^T\mathbf{x}(0)$ is the projection of the initial state $\mathbf{x}(0)$ onto the eigenbasis of $\mathbf{A}$, and the matrix exponential $\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{\Lambda})\right)$ applies exponential attenuation to each frequency component of the knowledge graph, ensuring that higher frequency components decay more rapidly over time.

1.2 Derivation of Equation (3)

To approximate the exponential matrix function in Equation (2), we utilize a first-order Taylor expansion with respect to τ. The general form of a Taylor expansion for a matrix exponential function is:

$$\exp(\mathbf{M}) \approx \mathbf{I} + \mathbf{M} + O(\mathbf{M}^2) \tag{10}$$

where $\mathbf{M}$ is the matrix inside the exponential. In our case, $\mathbf{M} = -\tau\alpha(\mathbf{I} - \mathbf{A})$. Applying this to the exponential function in Equation (2), we have:

$$\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{A})\right) \approx \mathbf{I} - \tau\alpha(\mathbf{I} - \mathbf{A}) + O(\tau^2) \tag{11}$$

Expanding this expression:

$$\mathbf{I} - \tau\alpha(\mathbf{I} - \mathbf{A}) = \mathbf{I} - \tau\alpha\mathbf{I} + \tau\alpha\mathbf{A} \tag{12}$$

Now, we can write this as:

$$\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{A})\right) \approx (1 - \tau\alpha)\mathbf{I} + \tau\alpha\mathbf{A} + O(\tau^2) \tag{13}$$

Thus, the matrix exponential $\exp\left(-\tau\alpha(\mathbf{I} - \mathbf{A})\right)$ can be approximated by the expression on the right-hand side, which is the first-order Taylor expansion in τ. The $O(\tau^2)$ term represents the higher-order terms that are neglected in this approximation. This approximation significantly reduces the computational complexity when dealing with large graphs, as we avoid computing the full matrix exponential, which is computationally expensive for large-scale data.

1.3 Derivation of Equation (13)

The forward diffusion process of standard Gaussian diffusion process starts with the original data $\mathbf{x_0}$ and progressively adds Gaussian noise at each time step t. At each step t, the data is smoothed by adding noise, moving from the original data towards pure noise at step T. The forward process is modeled as a Markov process, where the distribution at step t is conditioned on the data at step $t-1$:

$$q(\mathbf{x_t}|\mathbf{x_{t-1}}) = \mathcal{N}(\mathbf{x_t}; \sqrt{1-\beta_t}\mathbf{x_{t-1}}, \beta_t\mathbf{I}) \tag{14}$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

Here, β_t represents the variance of the noise added at each time step t. This process can be recursively formulated as:

$$q(\mathbf{x_t}|\mathbf{x_0}) = \mathcal{N}\left(\mathbf{x_t}; \sqrt{\bar{\alpha}_t}\mathbf{x_0}, (1-\bar{\alpha}_t)\mathbf{I}\right) \tag{15}$$

where $\bar{\alpha}t = \prod s = 1^t \alpha_s$ and $\alpha_t = 1 - \beta_t$. This equation models how the data is progressively corrupted with noise as t increases.

The reverse process aims to recover the original data from the noisy data at each time step by learning how to progressively remove the noise. The reverse process can be expressed as a conditional probability distribution, which is denoted as $q(\mathbf{x_{t-1}}|\mathbf{x_t})$. This distribution represents the probability of obtaining $\mathbf{x_{t-1}}$ given $\mathbf{x_t}$, the noisy data at time step t. In standard Gaussian diffusion process, the reverse process is modeled as a Gaussian distribution:

$$q(\mathbf{x_{t-1}} \mid \mathbf{x_t}, \mathbf{x_0}) = \mathcal{N}\left(\mathbf{x_{t-1}}; \mathbf{F_{t-1}}\mathbf{x_0}, \sigma^2_{t-1}\mathbf{I}\right) \tag{16}$$

where $\mathbf{F_{t-1}}$ is a learned function that maps the noisy data $\mathbf{x_t}$ at step t to the original data $\mathbf{x_0}$. The reverse process can be interpreted as a process of denoising, where each successive step attempts to reduce the noise added during the forward diffusion process.

Considering the real-time requirements of recommendation systems, we employ a deterministic reverse process to enhance efficiency. To achieve this, an important assumption is introduced to the reverse process: instead of directly learning the noise distribution at each time step, we assumes that the reverse process is implicitly parameterized using a deterministic trajectory that aligns more closely with the data distribution. This assumption is particularly useful for making the reverse process more efficient. The reverse process distribution can be rewritten as:

$$q(\mathbf{x_{t-1}}|\mathbf{x_t}, \mathbf{x_0}) = \mathcal{N}\left(\mathbf{x_{t-1}}; \mathbf{F_{t-1}}\mathbf{x_0} + \sqrt{\sigma^2_{t-1} - \sigma^2_{t-1}}\boldsymbol{\epsilon_{t-1}}, \sigma^2_{t-1|t}\mathbf{I}\right) \tag{17}$$

where$\mathbf{F_{t-1}}\mathbf{x_0}$ represents the predicted state of $\mathbf{x_{t-1}}$ at time step $t-1$, based on the model's learned parameters. $\boldsymbol{\epsilon_{t-1}}$ represents the noise term, capturing the difference between the smoothed data and the predicted data. $\sigma^2_{t-1|t}$ is the noise variance at each step, which can be adjusted or learned during training. This equation encapsulates the deterministic reverse process, where the model predicts the original data $\mathbf{x_0}$ and the noise $\boldsymbol{\epsilon_{t-1}}$ at each time step. The deterministic reverse process is modeled by a Gaussian distribution with the mean $\mathbf{F_{t-1}}\mathbf{x_0}$ and variance $\sigma^2_{t-1|t}$. The noise term $\boldsymbol{\epsilon_{t-1}}$ accounts for the randomness introduced by the forward smoothing process. By progressively refining the predicted data, the model recovers the original knowledge distribution. The reverse process is thus a denoising process where the model learns to predict the parameters of the Gaussian distribution at each step, effectively removing the noise and recovering the original data distribution.

2 Time & Space Complexity

2.1 Time Complexity

The total time cost of our proposed framework mainly comes from three parts: (1) the forward smoothing process, (2) the reverse refinement process, and (3) the continuous–discrete mode adapter. In the forward smoothing phase, the diffusion subprocess incurs a computational cost of $O(T|\mathcal{E}|^2)$, where $|\mathcal{E}|$ is the number of entities in the knowledge graph and T is a small, constant number of diffusion iterations. The convolution subprocess, which implements a heterogeneous graph convolution network with L layers, requires a complexity of $O(L|\mathcal{T}|d)$, where $|\mathcal{T}|$ is the number of triples and d s the embedding dimensionality. Consequently, the overall time complexity of the forward-smoothing process is $O(T|\mathcal{E}|^2 + L|\mathcal{T}|d)$. Since the reverse process is deterministic and involves no sampling, it incurs a computational cost of $O(T|\mathcal{E}|)$, which is minor. The continuous–discrete mode adapter first constructs a binary mask for each candidate triple and applies it to the adjacency matrix, an operation that costs $O(|\mathcal{T}|)$. Next, it executes L graph-convolution layers. Each layer performs a sparse matrix–vector multiplication that costs $O(|\mathcal{T}| + |\mathcal{E}|)$. Finally, the BPR loss is evaluated over $|D|$ ampled triplets, each requiring a dot product of two d-dimensional vectors, which costs $O(|D|d)$. Consequently, the overall time complexity is $O(|\mathcal{T}| + L(\mathcal{T} + |\mathcal{E}|) + |D|d)$. Thus, the model's runtime scales linearly with the number of nodes and edges, matching the efficiency of state-of-the-art baselines.

2.2 Space Complexity

The dominant term in the space requirement of our method stems from the node embeddings $E \in \mathbb{R}^{(|U|+|\mathcal{E}|)\times d}$, where $|U|$ is the number of users and $|\mathcal{E}|$ is the total number of entities (items are included because $I \subseteq \mathcal{E}$). A secondary contribution comes from the item-level retention and completion-rate vectors $\rho^-, \rho^+ \in \mathbb{R}^I$, which require only an $O(I)$. Thus, the overall space complexity is $O(|U| + |\mathcal{E}|) \times d + I)$, which is comparable to that of state-of-the-art baselines.